FSP Talent

Платформа подбора ИТ-специалистов с подтверждённым грейдом

ЛЦТ-2026, спецтрек ФСП · МГТУ ГА

Сопроводительная документация

0. Краткое описание подхода

Задача. Платформа подбора ИТ-специалистов, где работодатель сам выходит на кандидата в категоризированной базе и приглашает его с указанием зарплаты.

Механика подбора — «заявка на команду». Работодатель описывает потребность: специализацию, грейд, навыки с весами, зарплату от–до с пометкой «гросс» или «нетто», формат работы. Система показывает подходящие категории («специализация + грейд») с числом подтверждённых кандидатов и ранжированную подборку. У каждого места в подборке есть объяснение «почему он здесь»: какие навыки совпали, какой грейд подтверждён тестом и на каком процентиле категории, какие результаты ФСП учтены. Работодатель отправляет адресное приглашение с зарплатой; контакты кандидата открываются только после того, как кандидат принял приглашение или сам откликнулся на вакансию.

Механика тестирования. Грейд кандидата подтверждается только экспресс-тестом по категории (10 заданий, 5 минут). Задания — параметрические шаблоны: каждый шаблон по сиду кандидата генерирует свои числа, имена, фрагменты кода и порядок вариантов, правильный ответ вычисляет код. Сложность задаёт шаблон, а не случайность, поэтому разные варианты сопоставимы. Уровень кандидата оценивается по модели Раша, задания подбираются адаптивно. Слитые ответы бесполезны: у другого кандидата другие варианты. Это проверено симуляцией (раздел 4).

Почему так. Резюме можно приукрасить, а одинаковый для всех тест быстро утекает в сеть. Персональные варианты с калиброванной сложностью дают сопоставимую оценку и устойчивы к утечке. Результаты ФСП учитываются как дополнительный сигнал только внутри категории, поэтому кандидат без истории ФСП проходит весь путь наравне с другими.

1. Функциональная и компонентная архитектура

Компоненты

- **Браузер** — веб-приложение (Vite + React + TypeScript) с кабинетами кандидата и работодателя.
- **nginx** (контейнер `frontend`) — отдаёт статику и проксирует `/api` на бэкенд.
- **FastAPI** (контейнер `backend`, порт 8000) — API бизнес-логики, OpenAPI на `/docs`.
- **PostgreSQL 16** (контейнер `db`) — хранение; схема ведётся миграциями alembic, при старте бэкенда выполняются `alembic upgrade head` и загрузка демо-данных.

Всё поднимается одним `docker-compose.yml`. GPU и LLM в обязательном пути не используются.

Модули бэкенда

Три модуля без взаимных импортов — обмен только словарями и pydantic-схемами, роутеры FastAPI тонкие:

Модуль	Что делает
`testing`	банк шаблонов заданий, сборка попытки, адаптивный выбор, оценка θ по Рашу, грейд, кулдаун
`profile`	внешние учётные записи (FSP ID) и подтверждённые результаты из источников
`matching`	скор кандидата для потребности, объяснение, ранжирование, базовая линия

Ядро `testing` и `matching` — чистые функции без БД, поэтому одни и те же функции вызываются из API и из скриптов валидации.

Роли и права

Роль	Может
Кандидат	профиль, согласие на обработку ПДн, привязка FSP ID, выбор специализации и грейда, тест, приглашения, отклики
Работодатель	профиль компании, потребности и вакансии, категории и подборка, карточки кандидатов без контактов, приглашения

Контакты кандидата работодатель видит только при принятом приглашении или собственном отклике кандидата. Чужие данные по прямому адресу недоступны (403).

2. Механика тестирования и устойчивость к утечке заданий

Категория и грейд

- Категория = специализация (`backend`, `frontend`, `bas` — БАС-разработчик) + грейд (`junior`, `middle`, `senior`).
- Кандидат **сам выбирает** грейд, тест его подтверждает. Пороги уровня θ: junior ≥ −0,5; middle ≥ 0,5; senior ≥ 1,5.
- Статус: `confirmed` (подтверждён), `unconfirmed` (не подтверждён), `none` (тест не проходил). Не подтвердивший грейд кандидат не скрывается: он виден в подборке ниже подтверждённых (множитель 0,6 к скору). Принудительного понижения нет.
- Если θ выше порога следующего грейда, кандидату предлагается тест на повышение.

Кулдаун

Повторный тест на тот же или более высокий грейд — не раньше чем через 30 дней: за это время можно реально подтянуть навыки, а перебор попыток «до везения» теряет смысл. Тест на грейд ниже сразу после провала — без ожидания и без сжигания попытки. На демо срок сокращён флагом `DEMO_COOLDOWN_MIN`.

Шаблоны заданий

Шаблон = функция `generate(seed)` и метаданные `{id, spec, topic, b, kind}`. Сид = hash(candidate_id, attempt_id, template_id): у каждого кандидата свои числа, имена, фрагменты кода и порядок вариантов, правильный ответ вычисляет код. Виды заданий: выбор одного варианта, числовой ответ («что выведет код»), упорядочивание шагов.

Оценка по модели Раша и адаптивность

Вероятность верного ответа кандидата на задание:

$$P = 1 / (1 + \exp(-(\theta - b)))$$

где P — вероятность верного ответа (от 0 до 1); θ — уровень кандидата (логиты, безразмерная шкала); b — сложность шаблона (логиты, та же шкала).

Экспресс-тест — 10 заданий за 5 минут, темы покрывают специализацию. Первые три задания — около порога заявленного грейда, дальше адаптивно: верный ответ — следующий шаблон сложнее, неверный — проще. θ оценивается методом максимального правдоподобия при известных b, с ограничением [−3; 3]. Начальные b заданы экспертно и уточняются на симуляции (раздел 4).

Почему задания не утекают

1. **Значения уникальны для кандидата.** Слитый ответ относится к чужому варианту и не подходит к своему.
2. **Пул шаблонов на тему**, случайный выбор среди шаблонов близкой сложности — разные кандидаты получают разные наборы.
3. **Лимит времени** на задание и на тест мешает искать ответ на стороне.
4. **Статистика шаблона:** если доля верных ответов по шаблону резко растёт, шаблон помечается «скомпрометирован» и выводится из пула (концепция).

Количественная проверка — раздел 4, подраздел А4: сценарий «кандидат знает ответы всех вариантов, выданных 50 другим кандидатам» сравнивается с гипотетическим единым тестом для всех.

3. Механика подбора и категоризации

Обратный сценарий: работодатель находит кандидата

1. Работодатель создаёт потребность: специализация, грейд, навыки с весами 1–3, описание, **зарплата от–до в рублях с пометкой «гросс» или «нетто»** (без неё сохранить нельзя), формат работы.
2. Система показывает категории с числом кандидатов.
3. Ранжированная подборка с объяснением каждого места. Уточнение фильтров не сбрасывает подборку: исходный запрос хранится, фильтры — параметры запроса.
4. Адресное приглашение: сообщение, зарплата от–до, гросс или нетто, способ связи.
5. Кандидат принимает приглашение — работодателю открываются контакты. Фамилию, имя и места работы видно сразу, телефон, email и Telegram — только после согласия.

Жёсткий фильтр

Специализация и грейд из категории; флажок «соседние грейды» расширяет фильтр на ступень вверх и вниз.

Скор кандидата

score = 100 · (0,40 · stack_match + 0,30 · test_score + 0,15 · fsp_score + 0,10 · freshness + 0,05 · pref_match) · k

где: - score — итоговый скор кандидата для потребности, баллы от 0 до 100; - stack_match — доля веса требуемых навыков, которые есть в профиле (вес навыка задаёт работодатель), от 0 до 1; - test_score — процентиль θ кандидата внутри его категории (специализация + грейд), от 0 до 1; - fsp_score — вклад подтверждённых результатов ФСП, от 0 до 1: сумма очков (победа — 1, призёр — 0,7, участие — 0,2), дисциплина, близкая к специализации, — множитель 1,5; сумма 3 и больше даёт 1; при отсутствии истории ФСП — 0; - freshness — давность теста и регулярных заданий, от 0 до 1; - pref_match — совпадение формата работы (0,5; гибрид — 0,25) и зарплатных ожиданий с вилкой (0,5), от 0 до 1; - k — 1 для подтверждённого грейда и 0,6 для неподтверждённого (безразмерный).

Результаты ФСП влияют только на порядок **внутри** категории и не заменяют тест: все дисциплины равны, буст только за близость дисциплины к специализации (backend и frontend — «продуктовое программирование»; БАС-разработчик — «программирование беспилотных авиационных систем» и «программирование робототехники»).

Объяснение

Каждый ответ подбора содержит список факторов `{factor, weight, value, text}`, например: «Подтвердил middle (θ = 0,9, 82-й процентиль категории)», «Призёр ФСП по продуктовому программированию, 2026», «Нет истории ФСП — оценка по тесту и навыкам».

Кандидат без истории ФСП

Путь не блокируется ни на одном шаге: регистрация по email, профиль, тест, категория. Компонент ФСП у такого кандидата равен нулю, но он весит 15 %, а тест и навыки — 70 %, поэтому сильный кандидат без ФСП стоит выше слабого с ФСП (проверка — в разделе 4).

4. Процедура валидации и результаты

Процедура

1. Синтетическая база (`data/gen.py`, seed = 42): у каждого кандидата есть скрытая способность θ_true, которую не видят ни тест, ни подбор.
2. **Тестирование:** каждый кандидат проходит симуляцию теста дважды (ответ верен с вероятностью по модели Раша от θ_true); считаются повторяемость, точность против скрытой правды, разделяющая сила шаблонов, устойчивость к утечке.
3. **Подбор:** эталон пар «потребность — кандидат» размечен по скрытой правде, а не по формуле подбора (правило ниже), поэтому проверка не замкнута сама на себя. Сравнение с поиском по ключевым словам.
4. **Вторая независимая разметка** 200 пар (10 потребностей × 20 карточек) сделана вслепую: разметчик видел только карточки (специализация, заявленный грейд, навыки, стаж, результаты ФСП) без θ_true, без скора системы и без первой разметки. Разметку выполнила языковая модель в отдельном контексте; сомнительные случаи — `validation/stasya_doubts.md`. Согласие разметчиков — каппа Коэна.

Правило эталонной разметки

Labels are computed from the hidden truth, **not** from the matching formula, so the check is not circular.

- `grade_by_theta_true`: junior >= -0.5, middle >= 0.5, senior >= 1.5 (CONTRACT section 2); below -0.5 means "below junior".
- `coverage` = sum of need skill weights present in the candidate's skills / sum of all need skill weights.
- **2**: same spec, `grade_by_theta_true` == need grade, coverage >= 0.7.
- **1**: same spec, grade equal or adjacent (one step; "below junior" is adjacent to junior), coverage >= 0.4 (and not 2).
- **0**: everything else (not written to the file).

Why ranking by resume alone is not enough

- Experience and number of skills correlate with theta, but with noise (corr(theta, experience) = 0.69).
- `claimed_grade`: 70 % honest by theta, 20 % overstated by one step, 10 % understated.

Distributions (seed 42)

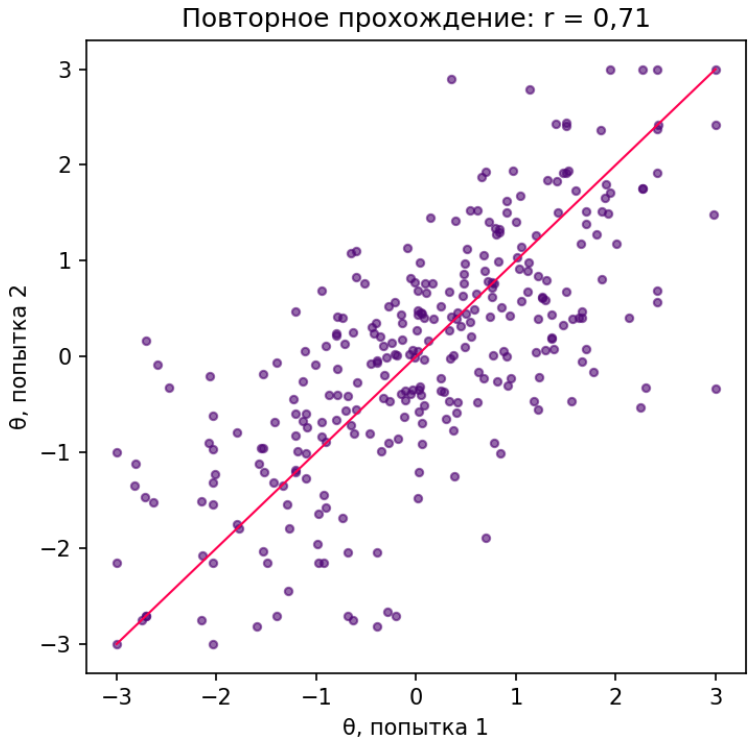
- spec: backend 135, frontend 135, bas 30; minors 16-17 y.o.: 15; with FSP history: 120 (40 %).
- grade by theta_true: below junior 98, junior 100, middle 80, senior 22. Claimed: junior 164, middle 105, senior 31.
- Pairs labeled 2 per need: from 0 to 23 (one need has no "2" — kept as is, reported honestly); labeled 1: 8 to 80.

Ядро: **real (backend/app/services)** Сгенерировано: `python -X utf8 validation/run_all.py`, данные seed = 42, 300 кандидатов, 30 потребностей.

A. Тестирование

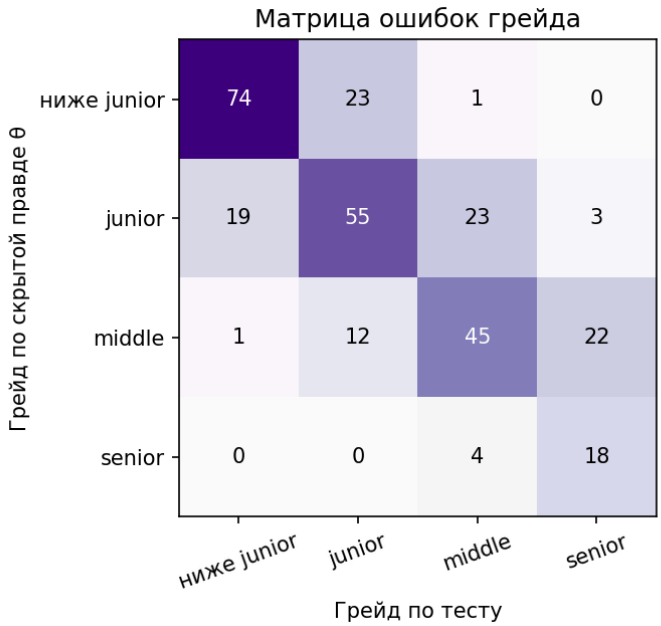
A1. Повторное прохождение (test-retest)

- Совпадение грейда по тесту между двумя попытками (разные варианты заданий): **56,3 %**
- Совпадение решения «подтверждён / не подтверждён»: **74,3 %**
- Корреляция Пирсона θ_1 и θ_2: **0,71**



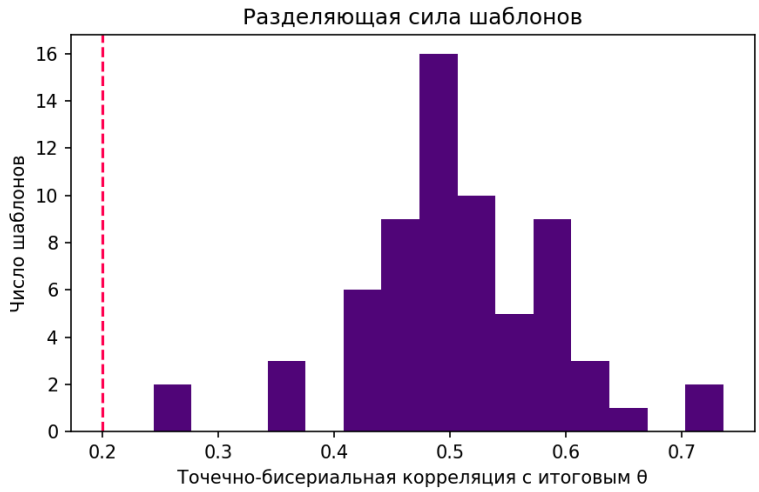
A2. Точность против скрытой правды

- Точное совпадение грейда: **64,0 %**; ошибка больше чем на ступень: **1,7 %**
- Решение о подтверждении заявленного грейда верно: **80,3 %**
- Средняя абсолютная ошибка θ: **0,52**



A3. Разделяющая сила шаблонов

- Шаблонов с ≥ 10 ответами: 66; медиана точечно-бисериальной корреляции: **0,50**
- Корреляция заявленной и уточнённой сложности b: **0,96**
- На доработку (корреляция < 0,2): **0**

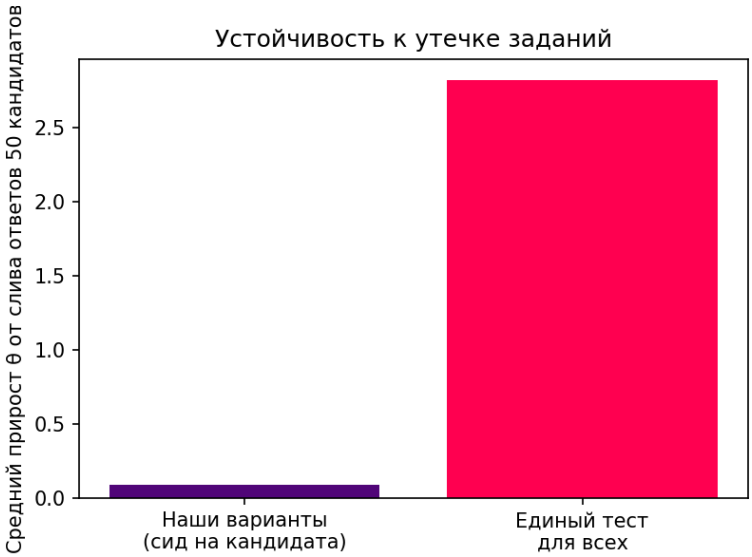


Шаблон	Ответов	r_pb	Доля верных, нижняя треть	Верхняя треть	b заявл.	b уточн.
bas.autopilot.modes	24	0,24	0,38	0,38	-1,10	-1,01
bas.mavlink.link_budget	18	0,26	0,17	0,50	1,50	2,43
frontend.web.debounce	87	0,35	0,28	0,52	1,10	1,34
backend.python.list_alias	58	0,35	0,16	0,53	1,50	2,04
backend.http.cache_codes	125	0,36	0,27	0,76	0,60	0,49
frontend.js.event_loop	89	0,41	0,14	0,59	1,20	1,68
backend.sql.isolation	57	0,42	0,21	0,68	1,60	1,54
backend.python.loop_mod	149	0,42	0,39	0,84	-0,90	-1,12
bas.mavlink.frame_size	22	0,42	0,29	0,86	1,20	0,71
frontend.css.specificity	162	0,42	0,22	0,69	0,30	0,16
backend.http.codes_basic	79	0,44	0,42	0,85	-1,40	-1,66
frontend.css.a11y	55	0,44	0,17	0,61	1,50	1,71
frontend.css.flex_grow	111	0,45	0,22	0,65	0,80	1,03
backend.sql.left_join	148	0,45	0,14	0,65	0,10	0,24
frontend.js.map_parseint	60	0,45	0,20	0,60	1,60	2,06

Показаны 15 шаблонов с наименьшей корреляцией; полный список — `results.json`.

A4. Устойчивость к утечке

- Доля пар кандидатов с одинаковым вариантом на одном шаблоне: **0,1 %**
- Слив ответов 50 других кандидатов: средний прирост θ **0,085**, максимум 0,84, прирост хоть какой-то у 17,0 % из 100 «сливщиков»
- Для сравнения, единый тест для всех: средний прирост θ **2,82**



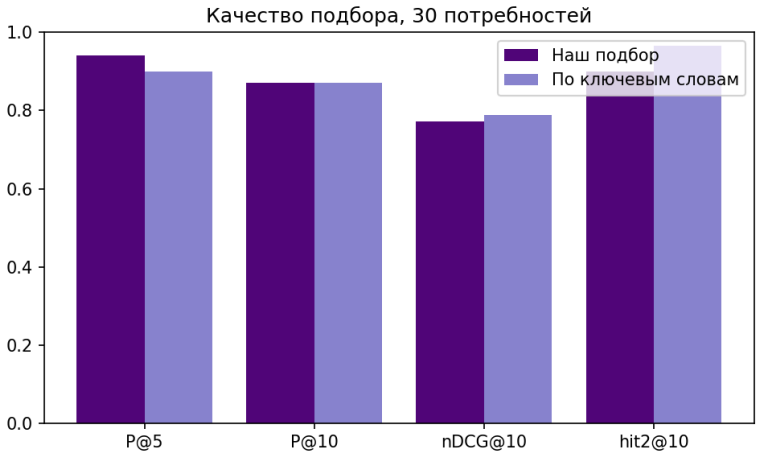
A5. Согласие разметчиков

- Пар: 200; совпадение: **77,5 %**; каппа Коэна: **0,64**; квадратично-взвешенная каппа: **0,78**

На 10 потребностях × 20 карточек	P@5	P@10	nDCG@10
Наш подбор (соседние грейды) против независимой разметки	0,98	0,85	0,81
Ключевые слова против независимой разметки	0,94	0,91	0,94
Наш подбор против разметки по скрытой правде	0,98	0,85	0,75

B. Подбор

Метод	P@5	P@10	nDCG@10	Есть «2» в топ-10
Наш подбор (грейд строго)	0,94	0,87	0,77	90,0 %
Наш подбор (соседние грейды)	0,97	0,94	0,72	96,7 %
По ключевым словам	0,90	0,87	0,79	96,7 %



Кандидаты без истории ФСП

- Доля без ФСП в топ-10: **53,6 %** при доле в базе 60,1 %
- Сильный кандидат без ФСП (разметка 2) выше слабого с ФСП (разметка 0): **91,4 %** из 3165 сравнений
- Пример: потребность 1 — кандидат 54 без ФСП на 25-м месте, кандидат 47 с ФСП (не подходит) на 36-м.

C. Производительность

- `rank` на 300 кандидатах: **0,6 мс**; на 3000: **9,3 мс** (медиана из 20 прогонов)

5. Интеграция с реестром ФСП и FSP ID (Keycloak)

Поток входа (OIDC, authorization code)

1. Кандидат нажимает «Войти через FSP ID»; бэкенд перенаправляет на `/auth/fsp/authorize` с `client_id`, `redirect_uri`, `state` и `scope=openid profile email`.
2. Keycloak ФСП аутентифицирует пользователя и возвращает код на `/auth/fsp/callback`.
3. Бэкенд обменивает код на `id_token` и `access_token`, проверяет подпись и `state`, читает claims.
4. Создаётся или находится внешняя учётная запись `ExternalIdentity(provider='fsp', subject=sub)`, уникальная по паре (provider, subject), и привязывается к пользователю.
5. По `subject` запрашиваются результаты участника из реестра ФСП и импортируются как подтверждённые результаты.

На демо Keycloak имитируется: те же эндпоинты и тот же обмен, данные берутся из `data/fsp_registry.json` (все кандидаты с FSP ID и 5 «свободных» участников для показа привязки).

Какие поля берём и зачем

Поле	Откуда	Зачем
`sub`	id_token	ключ внешней учётной записи
`name`, `email`	id_token	предзаполнение профиля (кандидат может исправить)
соревнование, дисциплина, год, место, роль, командный или личный зачёт	реестр ФСП	подтверждённый результат, компонент `fsp_score` и объяснение в подборке

Подтверждённый результат и источник

`VerifiedResult(source, external_id, event, discipline, year, place, role, team)` — подтверждённый результат из источника результатов. Поле `source` сейчас равно `fsp`; другие источники (олимпиады, хакатоны) добавляются без изменения схемы. Термин — «подтверждённый результат», не «достижение» (см. `CONTEXT.md`).

Замена имитации на боевой Keycloak

1. Зарегистрировать клиента в realm ФСП (confidential client, redirect URI стенда).
2. Указать в `.env` адрес realm, `client_id`, `client_secret`; адреса authorize, token и jwks берутся из `/.well-known/openid-configuration`.
3. Заменить источник результатов с файла на API реестра ФСП — функция `profile.import_results(source, claims, registry)` принимает данные и не зависит от транспорта.

Если истории ФСП нет

Кандидат выбирает «У меня нет истории ФСП» или привязывает FSP ID без результатов. Путь не блокируется, `fsp_score = 0`, в объяснении подборки — «нет истории ФСП», оценка идёт по тесту и навыкам.

6. Описание API

Префикс `/api`, авторизация — JWT в заголовке `Authorization: Bearer`. Полная спецификация — Swagger стенда `/docs` и файл `docs/openapi.json`.

Метод	Путь	Назначение
GET	`/api/health`	Проверка живости
POST	`/api/auth/register`	Регистрация по email
POST	`/api/auth/verify`	Подтверждение email кодом
POST	`/api/auth/login`	Вход по email и паролю
POST	`/api/auth/demo/{role}`	Вход демо-аккаунтом (для жюри)
GET	`/api/auth/fsp/authorize`	Связать профиль с FSP ID (начало OIDC)
GET	`/api/auth/fsp/callback`	Возврат от FSP ID (конец OIDC)
GET	`/api/me/profile`	Мой профиль
PUT	`/api/me/profile`	Изменить профиль, согласия и приватность
POST	`/api/me/resume/parse`	Разобрать PDF-резюме
GET	`/api/me/profile.pdf`	Стандартизированный PDF-профиль
GET	`/api/me/results`	Мои подтверждённые результаты
DELETE	`/api/me/fsp`	Отвязать FSP ID
GET	`/api/me/invitations`	Мои приглашения
GET	`/api/invitations/{inv_id}`	Открыть приглашение
POST	`/api/invitations/{inv_id}/accept`	Принять приглашение
POST	`/api/invitations/{inv_id}/decline`	Отклонить приглашение
POST	`/api/tests/start`	Начать тест на грейд
GET	`/api/tests/{attempt_id}`	Текущее состояние попытки
POST	`/api/tests/{attempt_id}/answer`	Ответить на задание
GET	`/api/tests/{attempt_id}/result`	Итог теста
GET	`/api/me/category`	Моя категория и история попыток
GET	`/api/employer`	Профиль компании
PUT	`/api/employer`	Изменить профиль компании
POST	`/api/needs`	Описать потребность
GET	`/api/employer/needs`	Мои потребности и вакансии
GET	`/api/searches/{need_id}`	Исходный запрос подборки (search_id = id потребности)
GET	`/api/needs/{need_id}`	Потребность
PUT	`/api/needs/{need_id}`	Изменить потребность / вакансию
POST	`/api/needs/{need_id}/publish`	Опубликовать как вакансию
GET	`/api/needs/{need_id}/categories`	Подходящие категории
GET	`/api/needs/{need_id}/matches`	Подборка кандидатов с объяснением
GET	`/api/candidates`	Поиск по банку кандидатов
GET	`/api/candidates/{cand_id}`	Карточка кандидата
POST	`/api/invitations`	Пригласить кандидата

Метод	Путь	Назначение
GET	`/api/employer/invitations`	Мои приглашения и их статусы
GET	`/api/needs/{need_id}/applications`	Отклики на вакансию
POST	`/api/applications/{app_id}/status`	Ответить на отклик
GET	`/api/vacancies`	Опубликованные вакансии
GET	`/api/vacancies/{need_id}`	Вакансия
POST	`/api/vacancies/{need_id}/apply`	Откликнуться
GET	`/api/me/applications`	Мои отклики и их статусы
GET	`/api/catalog`	Справочники
POST	`/api/tasks`	Опубликовать регулярное задание
GET	`/api/employer/tasks`	Мои регулярные задания
POST	`/api/tasks/{task_id}/close`	Снять задание
GET	`/api/tasks/{task_id}/answers`	Ответы на задание
POST	`/api/task-answers/{answer_id}/review`	Оценить ответ
GET	`/api/me/tasks`	Мои регулярные задания
POST	`/api/tasks/{task_id}/answer`	Ответить на задание

7. Библиотеки и компоненты с версиями

Бэкенд (`backend/requirements.txt`)

Пакет	Версия
fastapi	==0.143.0
starlette	==1.7.0
uvicorn[standard]	==0.54.0
SQLAlchemy[asyncio]	==2.1.4
asyncpg	==0.32.0
aiosqlite	==0.22.1
alembic	==1.20.0
pydantic	==2.14.0
email-validator	==2.3.0
passlib[bcrypt]	==1.7.4
bcrypt	==4.0.1
PyJWT	==2.15.1
python-multipart	==0.0.32
httpx	==0.28.1
pdfplumber	==0.11.10
reportlab	==5.0.1
pytest	==9.1.1
pytest-asyncio	==1.4.0

Фронтенд (`frontend/package.json`)

Пакет	Версия
@tanstack/react-query	5.59.0
react	18.3.1
react-dom	18.3.1
react-router-dom	6.26.2
recharts	2.13.0
@types/node	20.16.10
@types/react	18.3.11
@types/react-dom	18.3.0
@vitejs/plugin-react	4.3.2
autoprefixer	10.4.20
openapi-typescript	7.4.1
postcss	8.4.47
tailwindcss	3.4.13
typescript	5.6.2
vite	5.4.8

Docker-образы (`docker-compose.yml`)

- `postgres:16-alpine`
- `caddy:2.8-alpine`

Валидация и документация

- Python 3.12+, matplotlib, numpy (графики валидации); markdown (сборка документации).

8. Сборка, развёртывание и локальный запуск

Стенд для жюри

- Адрес: **https://81-26-180-5.sslip.io** (Swagger: `/api/docs`).
- Демо-входы — кнопки «Войти как кандидат» и «Войти как работодатель» на главной (без пароля, `POST /api/auth/demo/{role}`). Те же аккаунты с паролем: `candidate@demo.fsp` / `demo`, `employer@demo.fsp` / `demo`.
- На стенде включён демо-режим: код подтверждения email показывается на экране (имитация письма), кулдаун повторного теста на тот же грейд — 2 минуты вместо 30 дней.
- Сервер: Yandex Cloud, Ubuntu 24.04, 2 vCPU / 4 ГБ ОЗУ, HTTPS — Let's Encrypt через Caddy.

Локальный запуск одной командой

Нужны Docker и Docker Compose v2.

```
git clone https://github.com/Wit-Sky/fsp-lct2026
cd fsp-lct2026
cp .env.example .env        # можно пропустить: без .env работают значения по умолчанию
docker compose up -d --build
```

После старта: веб-приложение — http://localhost/, API — http://localhost/api, Swagger — http://localhost/api/docs. База PostgreSQL и демо-данные (300 синтетических кандидатов, банк шаблонов заданий, демо-аккаунты) создаются при первом запуске.

Сервисы `docker-compose.yml`:

Сервис	Что делает
`db`	PostgreSQL 16 (`postgres:16-alpine`), данные в томе `pgdata`
`backend`	FastAPI + SQLAlchemy 2 async, миграции alembic, порт 8000 только на 127.0.0.1
`frontend`	сборка React (Vite) и nginx: статика и прокси `/api` на бэкенд
`caddy`	HTTPS-прокси, включается профилем `https` (только на стенде)

Переменные `.env`:

Переменная	Назначение
`PUBLIC_URL`	внешний адрес стенда (для CORS и адреса возврата FSP ID)
`HTTP_PORT`	порт nginx фронтенда
`POSTGRES_PASSWORD`, `JWT_SECRET`	секреты; на стенде генерируются случайно и не покидают сервер
`DEMO_MODE`	1 — код подтверждения email показывается в ответе (имитация письма)
`DEMO_COOLDOWN_MIN`	кулдаун в минутах для демо; 0 — боевые 30 дней

Развёртывание на сервере (как поднят стенд)

1. Один раз: `deploy/bootstrap.sh` на чистой ВМ Ubuntu ставит Docker, добавляет swap 2 ГБ (сборка фронтенда на маленькой ВМ), создаёт пользователя `deploy` с ключом GitHub Actions и файл `.env` со случайными секретами. Домен без покупки — `<ip-через-дефисы>.sslip.io`.
2. Дальше автоматически: `.github/workflows/deploy.yml` при пуше в `main` проверяет сборку фронтенда, копирует код на ВМ (rsync), выполняет `docker compose up -d --build`, чистит старые образы (диск ВМ 9 ГБ) и ждёт ответа `/api/health`. Секреты репозитория: `DEPLOY_HOST`, `DEPLOY_SSH_KEY`.

Валидация и документация

```
python -X utf8 validation/run_all.py     # validation/results.md, results.json, графики в validation/img
python -X utf8 docs/build_pdf.py         # этот PDF
```

9. Концепции за рамками MVP

Защита от фиктивных вакансий

- Рейтинг ответов работодателя: доля приглашений, по которым работодатель ответил кандидату в срок.
- Доля приглашений без найма: аномально высокая — сигнал «сбор контактов», ограничение числа приглашений.
- Подтверждение компании по ИНН как добровольная метка доверия.
- Зарплата от–до обязательна — вакансию без денег опубликовать нельзя.

Интеграция с ATS

- Вебхуки статусов приглашений и откликов (sent, viewed, accepted, declined).
- Экспорт карточки кандидата (с его согласия) в JSON и HR-XML.

Регулярные задания от работодателей

Работодатель публикует задание по специализации со свободным ответом; ответы освежают профиль кандидата (компонент freshness) и дают работодателю повод для приглашения.

Кандидаты 16–17 лет и 152-ФЗ

- Дата рождения в профиле; младше 18 лет — обязательное согласие родителя, работодатель видит пометку «несовершеннолетний».
- Согласие на обработку персональных данных обязательно; кандидат управляет видимостью полей; контакты раскрываются только с его согласия.

Приложение А. Бэкенд: подробное описание

Автор — Иван Минаев; соответствует коду в `main` на 11.10.2026.

1. Компоненты и слои

Стек: Python 3.12, FastAPI, SQLAlchemy 2 (async) + asyncpg, PostgreSQL 16, alembic, Pydantic 2, JWT (PyJWT), bcrypt, pdfplumber (разбор резюме), reportlab (PDF-профиль). Один процесс uvicorn, один контейнер; БД — отдельный контейнер.

```
HTTP  →  routers/        auth, me, tests, employer, vacancies, tasks — валидация (schemas.py), права
(auth.py), ошибки (errors.py)
         ↓
         repo.py         загрузка из БД в обычные dict и сборка ответов
         ↓
         services/       testing · profile · matching — чистые функции, без БД и FastAPI
         ↓
         models.py + db.py  SQLAlchemy-модели, сессия; миграции — alembic/
```

Файл	Что делает
`app/main.py`	приложение FastAPI, CORS, `/api/health`, подключение роутеров (справочники `/api/catalog` — в `routers/vacancies.py`)
`app/config.py`	все настройки из переменных окружения (раздел 6)
`app/auth.py`	bcrypt-хэши паролей, JWT на 24 часа, зависимости «нужен кандидат / работодатель»
`app/fsp_oidc.py`	FSP ID как OIDC-клиент (Authorization Code Flow); `FSP_MODE=mock` — встроенная имитация провайдера
`app/errors.py`	единый формат ошибок `{detail, code}` для 401/403/404/409/422
`app/catalog.py`	справочники: специализации, грейды, пороги θ, навыки, дисциплины ФСП
`app/bank/`	банк параметрических шаблонов заданий (раздел 4)
`app/seed.py`	сид из `data/*.json` при пустой БД: 300 кандидатов прогоняются настоящим тестом
`app/pdf.py`	стандартизированный PDF-профиль кандидата (стек, роли, софт-скиллы, результаты ФСП)
`app/routers/tasks.py`	регулярные короткие задания от работодателей (раздел 5а)

Контакты кандидата (email, телефон, Telegram) отдаются работодателю только после принятия приглашения или собственного отклика кандидата; остальные поля — по настройкам приватности (`visible_fields`). Профиль несовершеннолетнего без согласия родителя работодателям не показывается.

2. Три модуля и их границы (ADR 0001)

Модуль	Отвечает за	Вход / выход
`services/testing.py`	банк шаблонов, адаптивная сборка теста, θ по Рашу, грейд, кулдаун, симуляция	состояние попытки — dict, хранится в `grade_attempts.items`
`services/profile.py`	импорт подтверждённых результатов из внешней учётной записи, `fsp_score`, видимость профиля, разбор резюме (ФИО, контакты, стек, грейд, роли, стаж, софт-скиллы)	dict результатов и профиля
`services/matching.py`	скор потребность × кандидат с объяснением, фильтры, категории, базовая линия «поиск по ключевым словам»	dict потребности и кандидатов

Правила: модули не импортируют друг друга, SQLAlchemy и FastAPI; обмениваются только словарями. Оценка ФСП попадает в `matching` данными (`fsp_score`, `fsp_explain`) или переданной функцией. Границы проверяет тест `test_module_boundaries` (разбор импортов через `ast`). Так каждый модуль переносится отдельно — в том числе в «Полёт БВС», где журнал полётов станет ещё одним источником `VerifiedResult`, а FSP ID — ещё одним провайдером входа.

3. Схема БД

Миграции: `0001_initial_schema`, `0002_profile_roles_soft_skills`, `0003_regular_tasks`, `0004_fsp_authorization_and_skip`; при старте контейнера выполняется `alembic upgrade head`, затем сид (только при пустой БД). URL берётся из `DATABASE_URL` в `alembic/env.py`. Типы обобщённые (`JSON`, `DateTime(timezone=True)`), поэтому та же схема работает на SQLite в тестах.

Таблица	Ключевые поля	Связи и ограничения
`users`	email, password_hash (bcrypt), role `candidate`/`employer`, email_verified, is_demo	email уникален
`external_identities`	provider (`fsp`), subject (FSP ID), claims (JSON)	→ users; unique(provider, subject)
`fsp_authorizations`	хэш одноразового кода, хэш cookie браузера, хэш OIDC state, сроки действия	→ users; код и state используются один раз
`candidate_profiles`	ФИО, дата рождения, контакты, город, формат, ожидания по зарплате, spec, skills, roles, soft_skills, consent_pd, parent_consent, visible_fields, fsp_skipped	PK = user_id → users
`verified_results`	source, external_id, event, discipline, year, place, role `winner`/`prize`/`participant`, team	→ users; unique(source, external_id, user_id)
`grade_attempts`	spec, claimed_grade, attempt_no, started_at, finished_at, theta, result, items (состояние попытки)	→ users
`categories`	spec, grade, status `confirmed`/`unconfirmed`, theta, tested_at, next_attempt_at, suggest_upgrade	→ users; unique(user_id, spec); индекс (spec, grade)
`employers`	name, inn, field, description, contacts	owner_id → users, уникален
`needs`	title, spec, grade, skills `[{name, weight}]`, вилка salary_from–salary_to, salary_type `gross`/`net`, work_format, published	→ employers; индекс (spec, grade); `published=true` — это вакансия
`invitations`	message, вилка зарплаты, contact_method, status `sent`/`viewed`/`accepted`/`declined`	→ employers, → users (кандидат), → needs (SET NULL)
`applications`	message, status	→ needs, → users; unique(need_id, candidate_id)
`tasks`	spec, grade (пусто — любой), title, text, kind `solve`/`approach`, active	→ employers; индекс spec
`task_answers`	answer, rating 0–3, feedback, reviewed_at	→ tasks, → users; unique(task_id, candidate_id)

Все внешние ключи — `ON DELETE CASCADE`, кроме `invitations.need_id`. Приглашение без зарплатной вилки не создаётся (422).

Связывание ФСП. Кандидат вызывает `POST /api/auth/fsp/start` с `Authorization: Bearer JWT`. Ответ `authorize_url` содержит случайный одноразовый код на 60 секунд, привязанный к кандидату и HttpOnly cookie браузера. `GET /api/auth/fsp/authorize?code=...` атомарно использует код и выдаёт непрозрачный OIDC state на 10 минут. Callback также одноразовый и требует ту же cookie. На HTTPS она имеет флаг Secure; SameSite=Lax допускает возврат от провайдера. JWT сессии в URL не передаётся; старый переход с `?token=`

больше не поддерживается. Ответы связывания имеют `Cache-Control: no-store` и `Referrer-Policy: no-referrer`. В БД хранятся хэши кодов, state и cookie.

Шаг без ФСП. `PUT /api/me/profile {"fsp_skipped": true}` сохраняет выбор между устройствами до первого теста. У существующих профилей после миграции значение false; связывание или отвязка ФСП очищает отметку.

Демовход. `POST /api/auth/demo/candidate` при каждом входе очищает только демо-кандидата: попытки, категорию, внешние связи и импортированные результаты, приглашения, отклики, ответы на задания и незавершённые сессии связывания. Профиль возвращается к исходным демоданным, ФСП ID снова свободен для выбора. При `DEMO_MODE=0` демовход отключён. Демо-работодатель и обычные пользователи не сбрасываются.

Счётчик подбора. В `/api/needs/{id}/matches` поле `total` — все кандидаты после фильтров, до ограничения `limit`; `items` — первые `limit` карточек. Веса и порядок ранжирования сохранены.

4. Банк заданий и устойчивость к утечке

Шаблон, а не вопрос. Каждое задание — функция-генератор (`app/bank/*.py`, декоратор `@template`) с калиброванной сложностью `b` по шкале Раша. Из сида генератор строит вариант: числа, таблицы, имена переменных, сценарий и порядок вариантов ответа. Правильный ответ вычисляется кодом генератора и в БД не хранится как константа.

Специализация	Темы (по 6 шаблонов, b от ≈ −1,4 до ≈ 1,6)
backend	python, http, sql, arch
frontend	js, react, css, web
bas	mavlink, nav, autopilot

Всего 66 шаблонов, типы ответа: выбор одного варианта, число, порядок шагов.

Сборка теста. 10 заданий за 5 минут, не больше 90 секунд на задание (опоздавший ответ считается неверным). Темы идут по кругу в порядке, своём для каждого кандидата. Первые 3 задания — около порога заявленного грейда, дальше сложность адаптируется: верно — цель +0,6, неверно — −0,6. Если время вышло или тест брошен, выданное задание без ответа засчитывается неверным, а каждое невыданное — неверным ответом на задание сложности порога заявленного грейда (в разбор по темам не попадает, в итоге — поле `unanswered`). Так пустая попытка не подтверждает грейд, а выход после пары верных ответов не завышает θ. Сид варианта: `sha256(candidate_id : attempt_no : template_id)` — у каждого кандидата и каждой попытки свои числа, а повтор воспроизводим (апелляция, аудит).

Оценка. θ — максимум правдоподобия модели Раша на сетке [−3; 3] с шагом 0,01 и слабой регуляризацией $N(0, 3^2)$, чтобы при всех верных/неверных ответах оценка была конечной. Пороги: junior −0,5, middle 0,5, senior 1,5. Грейд подтверждён, если θ ≥ порога заявленного; если θ дотягивает и до следующего — кандидату предлагается повыситься.

Кулдаун. Повтор на тот же или более высокий грейд — через 30 дней (на демо — 2 минуты, `DEMO_COOLDOWN_MIN`); после провала на грейд ниже — сразу. Ответ 403 содержит дату, когда можно снова.

Почему утечка мало помогает (валидация, 100 «сливщиков» получают ответы 50 других кандидатов):

Показатель	Значение
Пар кандидатов с одинаковым вариантом одного шаблона	0,1 %
Средний прирост θ от слива	0,085
Максимальный прирост θ от слива	0,84
Средний прирост θ при едином тесте для всех	2,82

В среднем слив помогает в 33 раза меньше, чем при едином тесте. Так стало после доработки банка 11.10: валидация показала, что у 15 шаблонов было всего от 4 до 72 различимых вариантов (числа брались из коротких списков), и слив давал до 2,52 по θ. Диапазоны чисел расширили, в шаблоны с выбором из готовых формулировок добавили вопросов, порядок дистракторов стал случайным. Теперь у всех шаблонов backend и frontend больше 150 различимых вариантов. Остались пять шаблонов БАС на знание фактов (режимы и параметры ArduPilot, failsafe, id сообщений MAVLink, GNSS) с 96–120 вариантами: новые факты в них добавляются только после вычитки экспертом БАС.

Другие показатели теста на настоящем ядре: точное совпадение грейда со скрытой правдой 64,0 %, ошибка больше чем на ступень 1,7 %, решение «подтверждён / нет» верно в 80,3 % случаев; повтор теста на других вариантах даёт корреляцию θ 0,71; медиана точечно-бисериальной корреляции шаблонов 0,50, шаблонов на доработку (< 0,2) нет.

5. Формула подбора

Жёсткий фильтр: специализация и грейд категории кандидата совпадают с потребностью (флажок «соседние грейды» — ±1), дополнительно: «только с историей ФСП», «обязательные навыки». Затем:

```
score = 100 × ( 0,40 × stack_match     доля весов требуемых навыков, которые есть у кандидата
              + 0,30 × test_score      процентиль θ внутри категории кандидата (специализация + грейд),
0..1
              + 0,15 × fsp_score       min(1, Σ очков / 3): победитель 1, призёр 0,7, участник 0,2;
                                       близкая дисциплина × 1,5; нет истории ФСП → 0
              + 0,10 × freshness       от более позднего из теста и ответа на регулярное задание: моложе
90 дней — 1,
                                       дальше линейно до 0,3 к году; без теста 0
              + 0,05 × pref_match )    формат работы (0,5; гибрид — частично 0,25) + ожидания в вилке
(0,5)
× 0,6, если грейд не подтверждён тестом
```

Близкие дисциплины ФСП: для backend и frontend — продуктовое программирование; для БАС — программирование БАС и робототехники. Каждый фактор возвращается в `explain: [{factor, weight, value, text}]` с текстом по-русски, например «Подтвердил middle: θ = 0,90, лучше 82 % категории» или «Нет истории ФСП — оценка только по тесту».

Отсутствие ФСП не исключает кандидата: вес ФСП 15 %, а тест и стек вместе — 70 %. По валидации сильный кандидат без ФСП стоит выше слабого с ФСП в 91,4 % сравнений; доля кандидатов без ФСП в топ-10 — 53,6 % при 60,1 % в базе. Для сравнения есть `keyword_baseline` — «поиск по резюме» (совпадение навыков + заявленный грейд). Скорость: `rank` на 300 кандидатах — 1,5 мс, на 3000 — 27 мс.

5a. Регулярные задания от работодателей

Работодатель публикует короткую задачу (`POST /api/tasks`): специализация, грейд (или любой), вид — «решить» или «предложить подход». Кандидату `GET /api/me/tasks` предлагает до 3 свежих заданий по его специализации и грейду, на которые он ещё не отвечал. Ответ — свободный текст (`POST /api/tasks/{id}/answer`, один на задание). Работодатель видит ответы на свои задания с именем и грейдом кандидата, но без контактов (`GET /api/tasks/{id}/answers`), ставит оценку 0–3 с комментарием (`POST /api/task-answers/{id}/review`); кандидат видит оценку у себя. Влияние на подбор: фактор актуальности считается от более позднего из теста и последнего ответа на задание, в explain — «Регулярное задание решено N дн. назад — профиль актуален». На грейд и θ задания не влияют: оценка работодателя субъективна, грейд подтверждает только тест. В сиде — три задания демо-работодателя (backend, frontend, БАС).

6. Переменные окружения

Все необязательны; значения по умолчанию подходят для демо. Шаблон — `backend/.env.example`.

Переменная	По умолчанию	Назначение
`DATABASE_URL`	`sqlite+aiosqlite:///./dev.db`	в compose: `postgresql+asyncpg://fsp:fsp@db:5432/fsp`
`DATA_DIR`	`<repo>/data`; в образе `/data`	откуда сид и имитация FSP ID берут `*.json` (том `./data:/data:ro`)
`JWT_SECRET`	строка для разработки	**сменить на стенде**, не короче 32 байт
`JWT_TTL_HOURS`	`24`	срок жизни токена
`CORS_ORIGINS`	`http://localhost:5173,http://localhost`	через запятую
`FRONTEND_URL`	`http://localhost:5173`	куда вернуть после привязки FSP ID
`COOLDOWN_DAYS`	`30`	кулдаун между попытками
`DEMO_COOLDOWN_MIN`	`2`	кулдаун в минутах для демо; `0` — использовать `COOLDOWN_DAYS`
`TEST_TIME_LIMIT_MIN`	`5`	время на весь тест
`ITEM_TIME_LIMIT_SEC`	`90`	время на одно задание
`DEMO_MODE`	`1`	код подтверждения email возвращается в ответе (имитация письма)
`SEED_ON_START`	`1`	загрузить сид при пустой БД
`FSP_MODE`	`mock`	`mock` — встроенная имитация; `oidc` — настоящий провайдер (Keycloak)
`FSP_ISSUER`	`http://localhost:8000/api/auth/fsp/mock`	адрес OIDC-провайдера
`FSP_CLIENT_ID`	`fsp-talent`	
`FSP_CLIENT_SECRET`	пусто	
`FSP_REDIRECT_URI`	`http://localhost:8000/api/auth/fsp/callback`	
`TEST_DATABASE_URL`	не задана	только для pytest: прогон на Postgres вместо SQLite

7. Проверка

`cd backend && pytest -q` — 44 теста, на SQLite и на PostgreSQL 16 (`TEST_DATABASE_URL=postgresql+asyncpg://…`). Среди них сквозной путь: регистрация → тест → категория → потребность → подборка с explain → приглашение → принятие → контакты видны; кулдаун; кандидат не видит чужие приглашения; кандидат без ФСП в подборке; уникальность и воспроизводимость вариантов; монотонность θ; разделение сильных и слабых в симуляции; границы модулей; разбор PDF-резюме (роли, софт-скиллы); регулярные задания (предложение, ответ, оценка, доступ только своей компании). Миграции проверены на PostgreSQL: `upgrade`/`downgrade`, перенос существующих профилей, `alembic check` без расхождений. Образ `backend/Dockerfile` собран и проверен на `postgres:16-alpine`: миграция, сид, `/api/health`, `/api/docs`.

8. Фактические версии (`pip freeze`, Python 3.12.10)

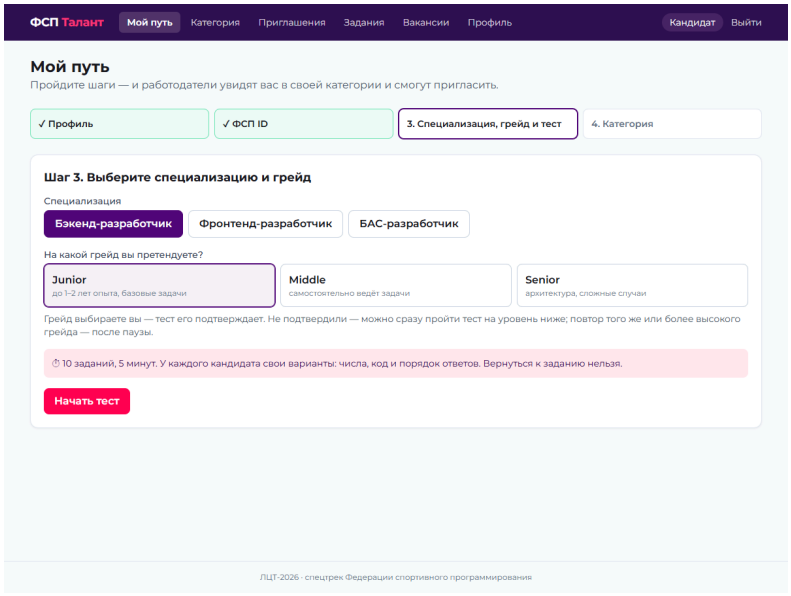
```
aiosqlite==0.22.1
alembic==1.20.0
annotated-doc==0.0.5
annotated-types==0.8.0
anyio==4.15.1
asyncpg==0.32.0
bcrypt==4.0.1
certifi==2026.7.22
cffi==2.1.1
charset-normalizer==3.5.2
click==8.5.0
```

```
colorama==0.4.6
cryptography==50.0.2
dnspython==2.9.0
email-validator==2.3.0
fastapi==0.143.0
greenlet==3.5.6
h11==0.16.0
httpcore==1.0.9
httptools==0.9.0
httpx==0.28.1
idna==3.20
iniconfig==2.3.1
Mako==1.4.3
MarkupSafe==3.0.4
opentelemetry-api==1.45.1
packaging==26.3
passlib==1.7.4
pdfminer.six==20260107
pdfplumber==0.11.10
pillow==12.3.0
pluggy==1.6.0
pycparser==3.11
pydantic==2.14.0
pydantic-settings==2.15.0
pydantic_core==2.50.0
Pygments==2.21.0
PyJWT==2.15.1
pypdfium2==5.14.0
pytest==9.1.1
pytest-asyncio==1.4.0
python-dotenv==1.2.4
python-multipart==0.0.32
PyYAML==6.0.3
reportlab==5.0.1
SQLAlchemy==2.1.4
starlette==1.7.0
typing-inspection==0.4.4
typing_extensions==4.16.0
uvicorn==0.54.0
watchfiles==1.3.0
websockets==17.2
```

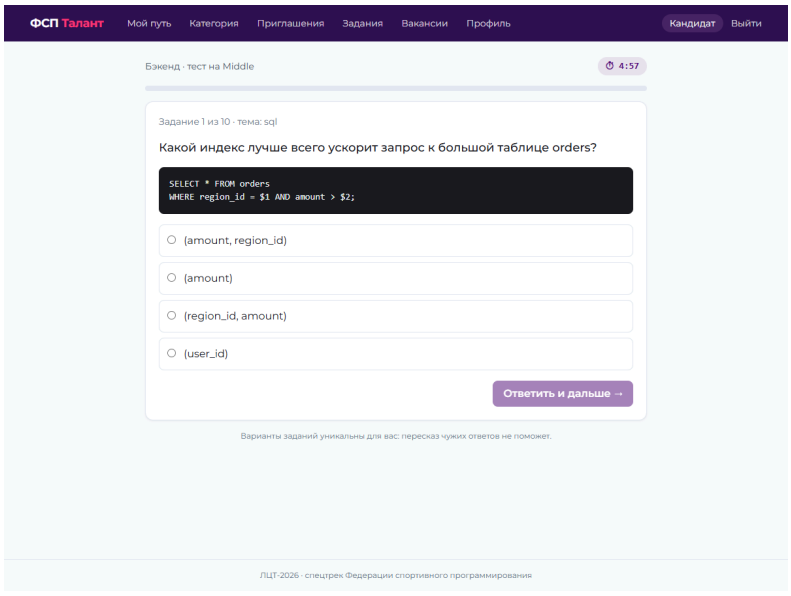
Приложение Б. Интерфейс

Снимки сделаны на стенде https://81-26-180-5.sslip.io 11.10.2026. Сквозной путь целиком показан в видео демонстрации.

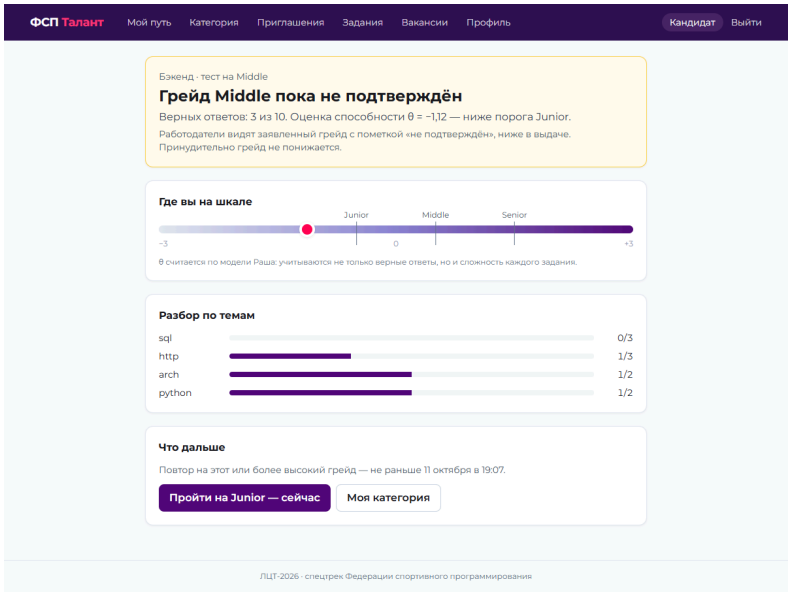
Кандидат: «Мой путь». Четыре шага: профиль, ФСП ID (или «У меня нет истории ФСП»), специализация с грейдом и тест, категория. Грейд кандидат выбирает сам, тест его подтверждает.



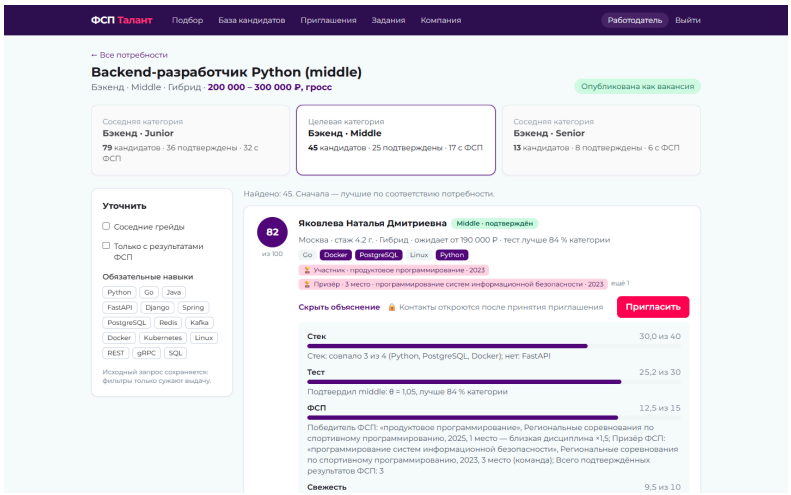
Экспресс-тест. 10 заданий, общий таймер 5 минут, вернуться к заданию нельзя. Числа, код и порядок вариантов у каждого кандидата свои.



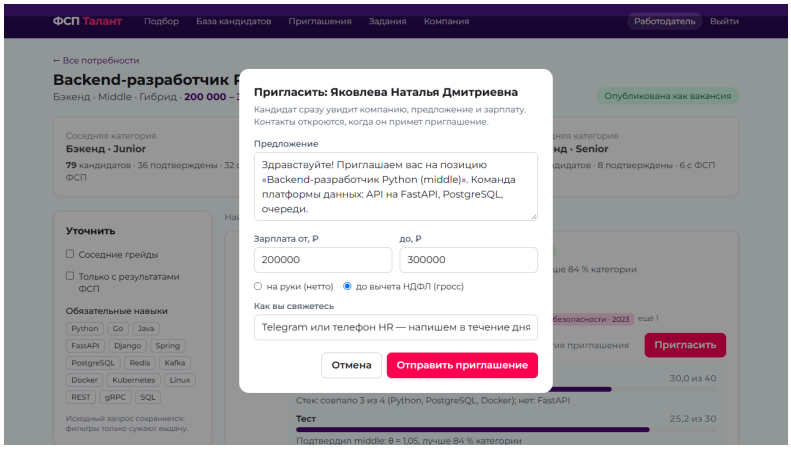
Итог теста. Оценка θ на шкале с порогами грейдов, разбор по темам, дата повтора и тест на уровень ниже сразу.



Работодатель: подборка по потребности. Целевая и соседние категории с числом подтверждённых кандидатов, фильтры, которые только сужают выдачу, и раскрытое «Почему в подборке» с вкладом каждого фактора.



Приглашение. Сообщение, зарплата от–до с пометкой гросс или нетто, способ связи. Контакты кандидата откроются после принятия.



Мобильная вёрстка (375 пикселей). Без горизонтальной прокрутки: меню и категории листаются лентой, фильтры сворачиваются.

ФСП Талант Работодатель Выйти

Подбор База кандидатов Приглашения

← Все потребности

Backend-разработчик Python (middle)

Бэкенд · Middle · Гибрид · **200 000 – 300 000 ₽, гросс**

Опубликована как вакансия

Целевая категория
Бэкенд · Middle
45 кандидатов · 25 подтверждены · 17 с ФСП

Соседняя катего
Бэкенд · Junio
79 кандидатов · подтверждены ·

Уточнить ▼

Найдено: 45. Сначала — лучшие по соответствию потребности.

82 из 100

Яковлева Наталья Дмитриевна

Приложение В. Сквозная проверка «как жюри»

Прогон 11.10.2026 на стенде, чистый профиль браузера (как инкогнито), экраны 1440 × 900 и 375 × 812 пикселей. Проверки доступа и обязательных полей дополнительно сделаны прямыми запросами к API от имени пользователей.

№	Что проверяли	Итог
1	Главная понятна за 10 секунд, демо-входы кандидатом и работодателем	прошло
2	Регистрация по email с кодом подтверждения (на демо код показывается)	прошло
3	Профиль: согласие на обработку данных обязательно, младше 18 лет — согласие родителя	прошло
4	Привязка FSP ID через имитацию; путь «У меня нет истории ФСП»	прошло (в первом прогоне путь без ФСП срабатывал только после обновления страницы; исправлено, повторный прогон 11.10 в 18:00)
5	Специализация и самостоятельный выбор грейда	прошло
6	Экспресс-тест 10 заданий / 5 минут; у двух аккаунтов разные варианты	прошло: разные таблицы, числа и порядок ответов
7	Не подтвердил — тест на уровень ниже сразу; повтор того же грейда запрещён с понятной датой	прошло
8	Потребность без зарплаты от–до и пометки гросс/нетто не сохраняется; «от» больше «до» — тоже	прошло
9	У каждого кандидата «Почему в подборке»; кандидат без ФСП не выглядит неполным; неподтверждённый грейд ниже	прошло
10	Фильтры меняют выдачу и возвращают исходную	прошло
11	Контакты скрыты, приглашение, принятие кандидатом, контакты видны работодателю	прошло
12	Кандидат не видит чужих приглашений по id, работодатель — чужих откликов	прошло (404 и 403)
13	Вакансии и отклик; мобильная вёрстка главной, теста и подборки	прошло, горизонтальной прокрутки нет
14	Сквозной путь целиком	сценарий видео демонстрации с паузами на подписи — 4 минуты; время человека, впервые видящего сервис, не замерялось